Project 4

Advanced Compilers

Released: May 28, 2026
Due: June 18, 2026

In previous project, we had found integer constants throughout the LLVM IR using Sparse Conditional Constants Propagation (SCCP). In this project, we will fold (propagate) constants using the `SimpleSCCP` analysis result. Thanks to SSA formed LLVM IR and LLVM's abundant APIs, manipulating variables is quite simple.

1 LLVM Pass/Analysis Manager

LLVM's middle-end, responsible for machine-independent optimizations, has two instances of `Pass Manager`[1]; `FunctionPassManager` and `ModulePassManager`. Those two pass managers are reponsible for function and module, respectively. Similarly, there are `FunctionAnalysisManager` and `Module AnalysisManager`, reponsible for the analyses over their IR units.

`PassManager` manages registered passes and calls them when they are needed as requested in optimization pipeline. While it is discouraged to call another pass inside one pass (i.e., nesting passes), a pass can freely request and use analyses results via `AnalysisManager`. However, a pass' modification (transformation) on IR can make the results of analyses incorrect. Therefore, a pass should invalidate corresponding analysis via analysis manager, if the pass had affetcted the result of other analyses.

For your information, LLVM had changed its pass (analysis) manager. Through these projects, we used and will use the new pass manager only.

2 SSA Form and Value Substitution

In SSA form, diffent values of the same variable are statically discriminated by their own name.

$$\begin{array}{l} a = 4 \\ x = a + 2 \\ a = 7 \\ y = a + 2 \end{array}$$

(a) Three-address code form

$$\begin{array}{l} a_0 = 4 \\ x_0 = a_0 + 2 \\ a_1 = 7 \\ y_0 = a_1 + 2 \end{array}$$

(b) SSA form

Figure 1: Example IR snippet

(a) is a code snippet in traditional normal (three-address code; TAC) form, and (b) is the same code but in SSA form. From the code, the variable x and y are being assigned the same expression $a + 2$. However, the value of x and y are 6 and 9, respectively. Although, x and y have different values, one cannot easily discriminate those values easily. However, in SSA form, those values are easily distinguishable, since their expressions are different. The key difference here is that a_0 and a_1

[1] There are more template instances of `PassManager`, used in back-end. (`llc`)

had their own name, unlike in (a). Therefore, when we substitute occurrences of a with their value (constant folding), we can easily decide the group of expressions to be substituted.

3 Code

3.1 `SimpleSCCPTransform`

Class for the costant propagation transformation pass.

3.2 `SimpleSCCPTransform::run`

Pass' main function. If the pass modified any of the IR, the pass should invalidate the previous analyses by returning `PreservedAnalyses::none`.

3.3 `SimpleSCCPTransform::foldConstants`

Core function which replaces *Use*s of the variables with known constants. One can easily remove the instruction from the basic block via `removeFromParent`. While substitution, one may need to replace branch (`br`) instructions. In LLVM, this instruction replacement or creation can be done through `IRBuilder`.

3.4 Workflow

1. Replace *Use*s of instructions analyzed as constants with the corresponding LLVM constants, and remove the replaced instructions when they are no longer needed.
2. If the condition of a conditional branch instruction is constant, replace the branch with an unconditional branch to the successor connected by the executable edge.
3. Remove basic blocks that do not have any executable incoming CFG edge after branch simplification.
4. For `PHI` instructions, remove incoming values from predecessors whose incoming CFG edges are not executable.

Tips

- Basic block should end with the terminator instruction.
- Basic blocks with zero reference count will be dropped automatically.
- Removing an iterator (or the corresponding element) while iterating over its range can invalidate the iterator.
- We encourage you to actively use vibe coding to reduce the burden of learning C++ syntax and LLVM APIs.
- One can find LLVM's implementation at `lib/Transforms/Scalar/SCCP.cpp`.

4 Input

As same as project 3, your pass will get an IR, which passed the `mem2reg` pass.

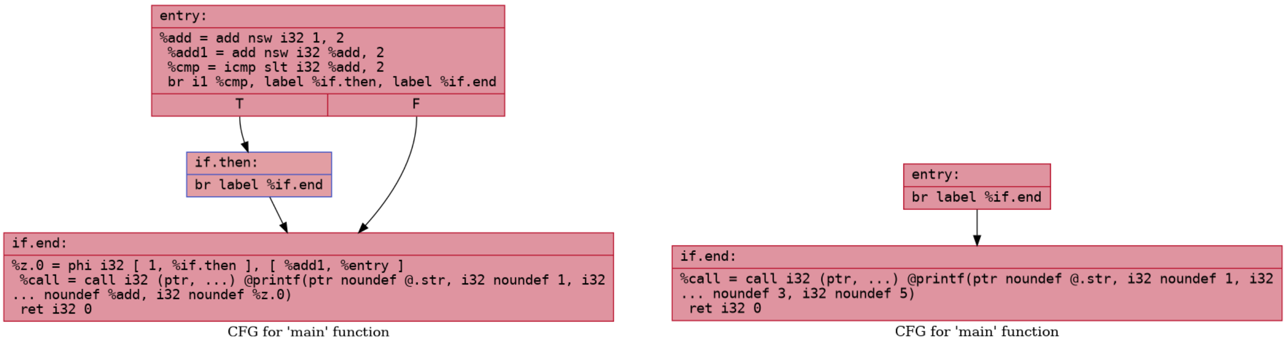
5 Objectives

1. Implement the `TODO` in `foldConstants` in `SimpleSCCP.cpp`.
   - Copy and paste your `visitPHINode`, `analyze`, and `visit` from Project 3.
   - Your result should be the same as LLVM's `sccp` result.
   - Transformed IR should be able to be compiled and executed normally.
2. Write a brief review of your implementation.
3. Prepare your own `testcase.c` where SCCP can be applied. You may refer to `test.c` as an example.
4. Run your `SimpleSCCP` transformation on your testcase and include the transformed result in your report.

6 Commands

- Build your plugin.
  ```
  cmake -S . -G Ninja -B build
  ```
- Generate IR from the source.
  ```
  clang -O0 -S -emit-llvm -Xclang -disable-O0-optnone \
    -Xclang -disable-llvm-passes -fno-discard-value-names \
    test.c -o test.ll
  ```
- Run the `mem2reg` pass.
  ```
  opt -S -passes='mem2reg' ./test.ll -o input.ll
  ```
- Call your pass from `opt`.
  ```
  opt -S -load-pass-plugin build/lib/libSimpleSCCP.so \
    -passes='simple-sccp' ./input.ll -o out.ll
  ```
- Run `verify`.
  ```
  opt -S -passes='verify' out.ll
  ```
- Generate IR with LLVM's `sccp`.
  ```
  opt -S -passes=sccp ./input.ll -o llvm_out.ll
  ```
- Compile the output IR.
  ```
  clang ./out.ll -o test
  ```

7 Example



(a) CFG of `input.ll` (b) CFG of `out.ll`

Figure 2: Example CFGs before and after `SimpleSCCP`.

8 Submission

- Submit a single zip archive to eTL.
- The archive must be in the `.zip` format. Other archive formats such as `.tar`, `.tar.gz`, `.tgz`, or `.7z` will not be accepted.
- The zip archive must contain exactly the following three files:
  - `SimpleSCCP.cpp`: your implementation.
  - `report.pdf`: a brief report that reviews your implementation and includes the transformed result for your own testcase. The report should discuss how your code performs constant folding and show the `SimpleSCCP` transformation result for your testcase.
  - `testcase.c`: a source code example where SCCP can be applied.